ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP E1–E3

SO SÁNH CÁC MÔ HÌNH PHÂN LOẠI ẢNH TRÊN TẬP DỮ LIỆU MNIST

Học phần: Học sâu và ứng dụng trong thị giác máy tính

Mã học phần: CO5085

Học kỳ: HK261, năm học 2026–2027

Học viên: Ngô Duy Bảo

Mã số học viên: 2570395

Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh

Mã nguồn: https://github.com/baond12/Deep_Learning_Course_HK261

Bản nháp phục vụ người học kiểm tra và chỉnh lý

MỤC LỤC

TÓM TẮT....3
CHƯƠNG 1. MỤC TIÊU VÀ THIẾT KẾ THỰC NGHIỆM CHUNG....4
CHƯƠNG 2. E1 – SOFTMAX, MLP VÀ CNN....6
CHƯƠNG 3. E2 – TỰ CHÚ Ý NHIỀU ĐẦU VÀ TOKEN HÓA ẢNH....9
CHƯƠNG 4. E3 – LSTM VÀ GRU TRÊN CHUỖI ẢNH....12
CHƯƠNG 5. SO SÁNH TỔNG HỢP VÀ THẢO LUẬN....16
KẾT LUẬN....19
TÀI LIỆU THAM KHẢO....20
PHỤ LỤC A. MINH CHỨNG MA TRẬN NHẦM LẪN....21
PHỤ LỤC B. TÁI LẬP VÀ KHAI BÁO HỖ TRỢ AI....25

TÓM TẮT

Báo cáo nghiên cứu thực nghiệm ba nhóm mô hình phân loại ảnh chữ số viết tay trên MNIST theo yêu cầu E1–E3 của học phần CO5085. E1 đối chiếu bộ phân loại tuyến tính Softmax, mạng truyền thẳng nhiều lớp (MLP) và mạng tích chập (CNN). E2 hiện thực cơ chế tự chú ý nhiều đầu (Multi-Head Self-Attention, MSA) bằng các phép toán PyTorch cơ bản và bằng mô-đun chuẩn, đồng thời so sánh ảnh được token hóa theo ô và theo hàng. E3 chuyển ảnh thành chuỗi để huấn luyện mạng bộ nhớ dài–ngắn hạn (LSTM) và đơn vị hồi quy có cổng (GRU), rồi đối chiếu với MLP/CNN.

Các thí nghiệm dùng chung bộ dữ liệu và một quy trình huấn luyện: 54.000 ảnh huấn luyện, 6.000 ảnh xác thực, 10.000 ảnh kiểm tra; 20 epoch, kích thước lô 16, Adam và cùng seed 42. Có 12 cấu hình được ghi nhận, mỗi cấu hình có đầy đủ lịch sử 20 epoch. Kết quả tốt nhất trên tập kiểm tra thuộc CNN đối chứng trong E3 (99,20%); CNN E1 đạt 99,17%. Trong E2, cách biểu diễn theo hàng đạt 97,40%, cao hơn hai bản theo ô trong lần chạy được cung cấp. Trong E3, GRU theo hàng đạt 98,87% với 61.962 tham số. Những quan sát này đúng trong phạm vi cấu hình và tập MNIST, chưa phải kết luận tổng quát giữa các họ mô hình [1]–[6].

Từ khóa: MNIST; phân loại ảnh; Softmax; MLP; CNN; multi-head self-attention; LSTM; GRU; biểu diễn token; ma trận nhầm lẫn.

CHƯƠNG 1. MỤC TIÊU VÀ THIẾT KẾ THỰC NGHIỆM CHUNG

1.1. Bài toán và mục tiêu đối chiếu

Cho ảnh $x \in \mathbb{R}^{1x28x28}$ biểu diễn một chữ số viết tay, nhiệm vụ là dự đoán nhãn $y \in \{0, 1, \ldots, 9\}$. Cả ba bài tập được tổ chức trên cùng một bài toán để tập trung vào khác biệt về biểu diễn đầu vào và kiến trúc: biến ảnh thành vector cho Softmax/MLP, giữ cấu trúc lân cận không gian cho CNN, chuyển ảnh thành token cho MSA và chuyển thành chuỗi cho LSTM/GRU. Yêu cầu của đề còn bao gồm vòng lặp huấn luyện tự viết, định lượng hiệu năng và phân tích lỗi, chứ không chỉ báo cáo độ chính xác cao nhất [1].

1.2. Dữ liệu và tiền xử lý

MNIST gồm ảnh xám 28 × 28 điểm ảnh thuộc 10 lớp. Bộ train gốc 60.000 ảnh được chia ngẫu nhiên với bộ sinh số khởi tạo seed = 42 thành 54.000 ảnh dùng cập nhật trọng số và 6.000 ảnh xác thực, tương ứng 90% và 10%; 10.000 ảnh test gốc được giữ riêng. Ảnh được chuyển thành tensor và chuẩn hóa theo $x' = (x - 0{,}1307)/0{,}3081$. Mã nguồn không thực hiện biến đổi tăng cường dữ liệu (augmentation) [2]–[4].

Bảng 1. Phân chia MNIST và vai trò trong đánh giá

Tập dữ liệu	Số lượng	Vai trò
Huấn luyện	54.000	Cập nhật trọng số qua lan truyền ngược
Xác thực	6.000	Theo dõi và chọn trạng thái tốt nhất
Kiểm tra	10.000	Đánh giá sau khi chọn mô hình

Việc chọn trọng số theo tập xác thực mà không dựa vào tập kiểm tra giúp giới hạn nguy cơ "học" theo test. Cùng một phép chia được sử dụng ở cả E1, E2 và E3, nhưng không nên đồng nhất một seed với độ ổn định đã được kiểm chứng qua nhiều lần chạy. Các mô hình lần lượt được khởi tạo trong mỗi chương trình; seed được đặt một lần trước vòng lặp mô hình, không reset độc lập trước mỗi mô hình [2]–[5].

1.3. Quy trình huấn luyện và lựa chọn mô hình

Bảng 2. Cấu hình huấn luyện ghi trong kết quả thực nghiệm

Thành phần	Giá trị ghi nhận
Epoch	20
Batch size	16
Bộ tối ưu	Adam
Learning rate	0,001

Thành phần	Giá trị ghi nhận
Weight decay	0,0001
Hàm mất mát	CrossEntropyLoss (nhận logits)
Seed	42
Thiết bị	CUDA (chưa có tên GPU cụ thể)
Lưu trạng thái	Sao chép trạng thái khi val accuracy tăng; nạp lại trước test

Ở mỗi mini-batch, chương trình lần lượt xóa gradient, tính logits, tính CrossEntropyLoss, gọi backward() và optimizer.step(). Sau mỗi epoch, mô hình chuyển sang chế độ đánh giá để tính loss/accuracy trên tập xác thực. Chương trình giữ bản sao trọng số trong bộ nhớ ở epoch có độ chính xác xác thực lớn nhất; trường hợp bằng nhau giữ epoch xuất hiện sớm nhất. Toàn bộ 20 epoch vẫn được thực hiện, không áp dụng dừng sớm, scheduler hay gradient clipping [2]–[4].

CrossEntropyLoss xử lý phần log-softmax ổn định về số học bên trong, vì vậy các model trả về logits, không đặt thêm softmax trước hàm mất mát. Khi suy luận, argmax(logits) và argmax(softmax(logits)) cho cùng nhãn dự đoán. Thời gian "training_time" bao gồm các lượt huấn luyện và xác thực, cùng thao tác lưu/nạp trạng thái tốt nhất trong hàm fit_classifier; không bao gồm đánh giá test hoặc tạo hình [2].

1.4. Chỉ số đánh giá và cách đọc hình

Độ chính xác (accuracy) là số dự đoán đúng chia tổng số ảnh. F1 macro là trung bình cộng F1 của từng lớp, vì vậy phản ánh hiệu năng theo lớp mà không chỉ dựa vào các lớp nhiều mẫu. Bên cạnh đó, báo cáo xét test loss, số tham số có requires_grad, thời gian huấn luyện và đường cong train/validation. Ma trận nhầm lẫn đặt nhãn thật theo hàng, nhãn dự đoán theo cột; các ô ngoài đường chéo cho biết dạng nhầm lẫn. Hình lỗi là tối đa 25 mẫu sai xuất hiện đầu tiên khi duyệt test, không phải lấy mẫu ngẫu nhiên [2], [5], [6].

Lưu ý khi đọc đường cong: độ chính xác huấn luyện được đo ở chế độ model.train(), còn xác thực ở model.eval(). Vì dropout chỉ hoạt động trong huấn luyện, có thể xuất hiện val accuracy lớn hơn train accuracy mà không tự nó chứng minh rò rỉ dữ liệu. Các nhận xét về dấu hiệu quá khớp chỉ dựa trên diễn biến loss và chênh lệch giữa hai tập trong lần chạy đã ghi nhận.

CHƯƠNG 2. E1 – SOFTMAX, MLP VÀ CNN

2.1. Thiết kế ba kiến trúc

Softmax Regression được xây dựng bằng Flatten rồi Linear(784, 10): $z = Wx + b$, với $x \in \mathbb{R}^{784}$ và $W \in \mathbb{R}^{10x784}$. Mô hình có $784 \times 10 + 10 = 7.850$ tham số. Đây là bộ phân loại tuyến tính trên không gian điểm ảnh gốc, không có các tầng học đặc trưng phi tuyến; đầu ra là logits để đưa vào CrossEntropyLoss [2], [7].

MLP nhận vector 784 chiều, đi qua Linear(784, 256) → ReLU → Dropout(0,2) → Linear(256, 128) → ReLU → Dropout(0,2) → Linear(128, 10). Số tham số lần lượt của ba tầng tuyến tính là 200.960, 32.896 và 1.290; tổng cộng 235.146. Nhờ ReLU, MLP xây dựng được ranh giới quyết định phi tuyến, nhưng phép làm phẳng khiến kiến trúc không khai thác trực tiếp quan hệ láng giềng giữa các pixel [2], [7].

CNN gồm hai khối Conv2d(1→32, 3×3, padding 1) → ReLU → MaxPool2d(2) và Conv2d(32→64, 3×3, padding 1) → ReLU → MaxPool2d(2). Kích thước tensor lần lượt là 1×28×28 → 32×14×14 → 64×7×7; sau Flatten, head Linear(3136, 128) → ReLU → Dropout(0,3) → Linear(128, 10). Bốn tầng có tham số 320, 18.496, 401.536 và 1.290, tổng 421.642. Tích chập chia sẻ trọng số giữa các vị trí và tổng hợp đặc trưng cục bộ, phù hợp với cấu trúc nét viết của MNIST [2], [8].

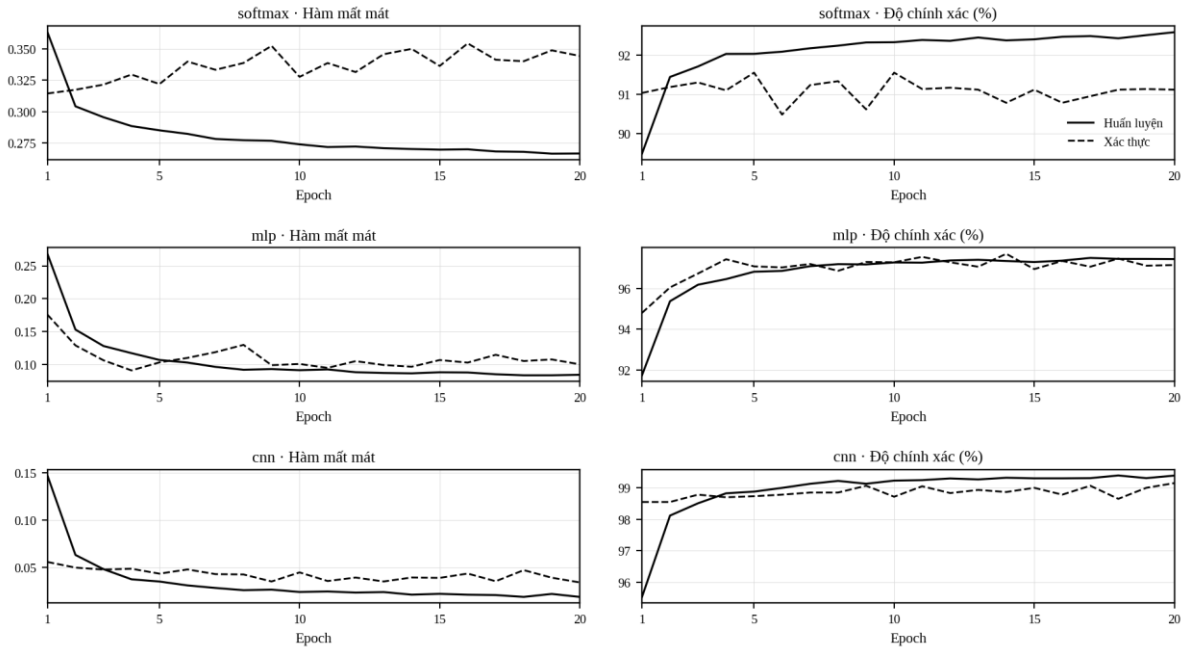
2.2. Kết quả định lượng

Bảng 3. Kết quả E1 trên tập kiểm tra và quy mô mô hình

Mô hình	Val tốt nhất	Test accuracy	Macro-F1	Tham số	Epoch tốt nhất
Softmax	91,55%	92,09%	0,919632	7.850	5
MLP	97,70%	97,82%	0,978016	235.146	14
CNN	99,15%	99,17%	0,991646	421.642	20

CNN có độ chính xác test 99,17%, cao hơn MLP 1,35 điểm phần trăm và cao hơn Softmax 7,08 điểm phần trăm; MLP đạt 97,82%, cao hơn Softmax 5,73 điểm phần trăm. Cùng chiều với accuracy, F1 macro tăng từ 0,919632 lên 0,978016 rồi 0,991646. CNN đổi lại cần số tham số lớn hơn và thời gian ghi nhận 504,7 giây, so với 471,3 giây của MLP và 418,4 giây của Softmax [5].

2.3. Diễn biến hội tụ

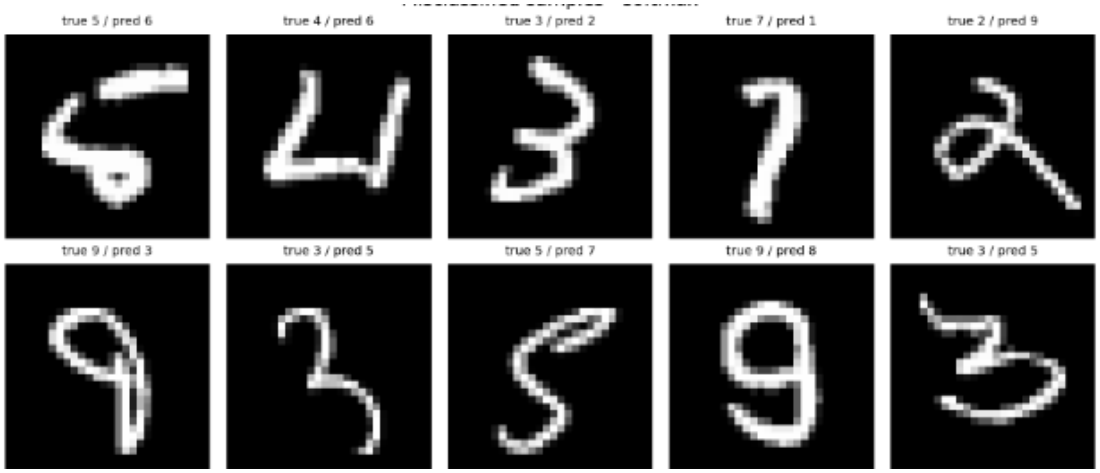


Hình 1. Đường cong loss và accuracy cho ba mô hình E1. Nét liền: huấn luyện; nét đứt: xác thực. Nguồn: dựng từ lịch sử 20 epoch [6].

Softmax giảm training loss từ khoảng 0,363 xuống 0,266 nhưng validation loss biến động quanh 0,32–0,35; best validation accuracy xuất hiện tại epoch 5. Điều này gợi ý năng lực mô hình tuyến tính đã chạm giới hạn đối với dữ liệu, đồng thời có khoảng cách train–validation. MLP cải thiện nhanh trong các epoch đầu, đạt best validation accuracy tại epoch 14. CNN duy trì validation loss thấp hơn, accuracy xác thực ổn định quanh 99% và đạt mức tốt nhất trong lịch sử ở epoch 20 [5], [6].

2.4. Phân tích lỗi

Ma trận nhầm lẫn E1 (Phụ lục A) cho thấy sai sót của Softmax phân tán hơn MLP/CNN. Ví dụ có 46 ảnh chữ số 5 bị dự đoán thành 3, 42 ảnh chữ số 2 thành 8 và 33 ảnh chữ số 8 thành 3; các giá trị được đọc trực tiếp từ ma trận gốc. Với CNN, các ô ngoài đường chéo nhỏ hơn đáng kể; một số nhầm lẫn vẫn còn, chẳng hạn 12 ảnh số 9 bị nhận thành 4 và 9 ảnh số 5 thành 3. Nhận xét này chỉ áp dụng cho ma trận của checkpoint được đánh giá, không giải thích chắc chắn nguyên nhân từng mẫu [5].



Hình 2. Mười mẫu sai đầu tiên được hiển thị của Softmax, trích từ ảnh lỗi gốc. Ký hiệu "true/pred" trên ảnh tương ứng nhãn thật/nhãn dự đoán [2].

Các chữ số có nét giao nhau, viết lệch hoặc có dáng gần lớp khác khiến việc phân tách bằng ranh giới tuyến tính khó hơn. Mẫu sai không đại diện cho toàn bộ phân phối dữ liệu vì chương trình giữ tối đa 25 lỗi đầu tiên theo thứ tự test; chúng đóng vai trò minh họa định tính, không được dùng để ước lượng tỷ lệ lỗi theo hình dạng [2].

CHƯƠNG 3. E2 – TỰ CHÚ Ý NHIỀU ĐẦU VÀ TOKEN HÓA ẢNH

3.1. Mục tiêu và biểu diễn đầu vào

E2 đánh giá hai câu hỏi: một hiện thực MSA dùng các phép toán PyTorch cơ bản có cho kết quả cùng mức với nn.MultiheadAttention hay không; và thay đổi đơn vị token từ ô ảnh sang hàng ảnh ảnh hưởng thế nào đến độ chính xác và chi phí trong cùng cấu hình nhỏ. Ba chương trình đối chiếu là custom_patch, pytorch_patch, custom_row; hai cách token hóa được hiện thực, thỏa mức bắt buộc của đề [1], [3].

Ở nhánh patch, Conv2d có kernel_size = stride = 7 chiếu ảnh 1×28×28 thành lưới 4×4 gồm 16 token, mỗi token 64 chiều. Ở nhánh row, Linear(28, 64) chiếu từng hàng ảnh thành 28 token. Cả hai thêm một token phân loại học được (CLS) và nhúng vị trí tuyệt đối học được; chuỗi đưa vào khối encoder có dạng B×17×64 đối với patch, B×29×64 đối với row. Sau encoder, đặc trưng CLS được chuẩn hóa và ánh xạ thành 10 logits [3], [9].

Bảng 4. Hình dạng token và cấu hình encoder của E2

Thành phần	Patch 7×7	Theo hàng
Token ảnh	16	28
Tổng token có CLS	17	29
Kích thước embedding	64	64
Số head	4	4
Độ rộng mỗi head	16	16
FFN	64 → 128 → 64	64 → 128 → 64
Số block	1 (pre-norm)	1 (pre-norm)

3.2. Công thức và cơ chế MSA

Với chuỗi $X \in \mathbb{R}^{BxNx64}$, các phép chiếu tuyến tính tạo $Q = XW_Q$, $K = XW_K$ và $V = XW_V$. Sau khi chia bốn head, mỗi Q_i, K_i, V_i có kích thước B×N×16. Ở head i, attention được tính bằng $A_i = \text{softmax}(Q_iK_i^T/\sqrt{16})$, đầu ra $H_i = A_iV_i$. Ghép bốn H_i và chiếu tuyến tính sẽ trở về B×N×64. Bản custom triển khai rõ reshape, permute, phép nhân ma trận và softmax, không gọi nn.MultiheadAttention; bản PyTorch sử dụng nn.MultiheadAttention(batch_first=True, need_weights=False) [3], [10].

Một Transformer Encoder Block sử dụng tiền chuẩn hóa: $X_1 = X + \text{Dropout}(\text{MSA}(\text{LayerNorm}(X)))$, $X_2 = X_1 + \text{FFN}(\text{LayerNorm}(X_1))$. FFN gồm hai phép

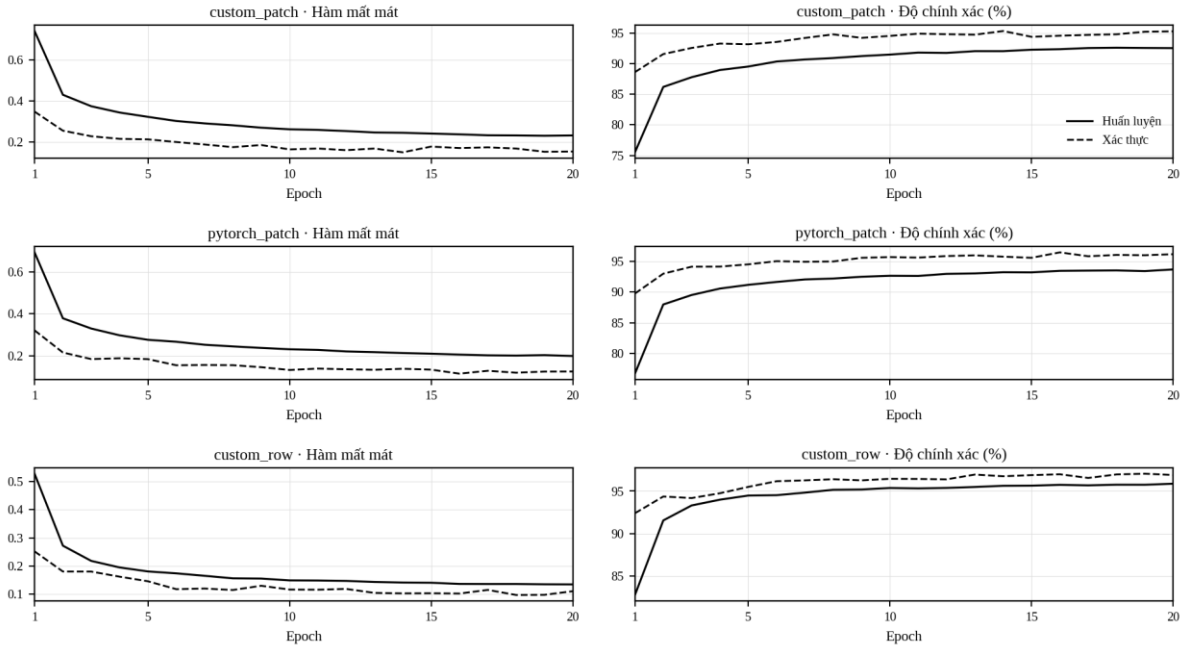
chiếu 64→128→64, kích hoạt GELU và dropout. Với token CLS, ma trận attention ở mỗi head có $17^2 = 289$ phần tử cho patch và $29^2 = 841$ phần tử cho row, nếu không tính chiều batch/head. Khác biệt bậc hai này mô tả riêng phần ma trận attention, không thể đồng nhất với tổng thời gian huấn luyện vì còn overhead, kernel và các tầng khác [3], [10].

3.3. Kết quả thực nghiệm

Bảng 5. Kết quả E2: MSA và biểu diễn token

Cấu hình	Test accuracy	Macro-F1	Tham số	Huấn luyện	Epoch tốt nhất
Tự viết · patch	95,63%	0,955523	38.602	660,6 s	14
PyTorch · patch	96,66%	0,966205	38.602	650,8 s	16
Tự viết · hàng	97,40%	0,973801	38.026	637,4 s	19

Hai mô hình patch cùng 38.602 tham số, nhưng pytorch_patch đạt 96,66% trên tập kiểm tra, cao hơn custom_patch (95,63%) 1,03 điểm phần trăm. So với custom_patch, custom_row đạt 97,40%, tăng 1,77 điểm phần trăm dù ít hơn 576 tham số. Thời gian đo được của custom_row là 637,4 giây, ngắn hơn custom_patch (660,6 giây); bởi vậy không thể suy ra chuỗi nhiều token hơn nhất thiết chạy chậm hơn trong thực nghiệm này [5].



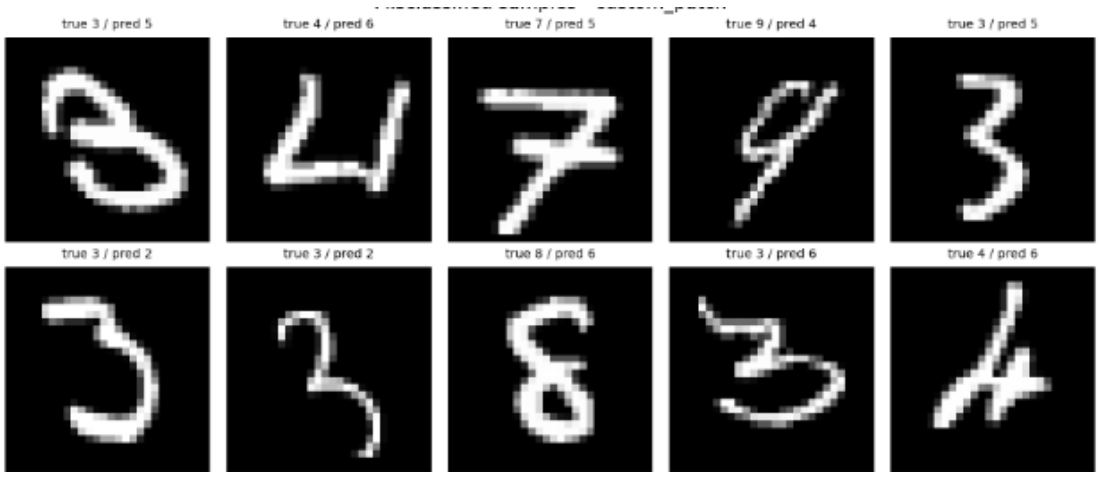
Hình 3. Đường cong huấn luyện và xác thực của ba cấu hình E2, dựng trực tiếp từ lịch sử thực nghiệm [6].

Các mô hình cùng giảm loss mạnh ở đầu quá trình và sau đó tiến tới vùng dao động nhẹ. Với custom_patch, validation accuracy duy trì cao hơn train accuracy khá rõ, phù hợp với khác biệt chế độ dropout đã nêu ở Chương 1. Cả ba mô hình được giữ trạng thái tại epoch tốt nhất theo độ chính xác xác thực, không mặc định sử dụng epoch 20 (lần lượt ở epoch 14, 16 và 19) [5], [6].

3.4. Giới hạn của phép đối chiếu MSA và phân tích lỗi

Hai nhánh patch không trùng hoàn toàn đường tính toán: bản tự viết có dropout ở đầu ra projection trước dropout của block, trong khi wrapper PyTorch không có lớp dropout tương ứng tại vị trí đó; cơ chế khởi tạo tham số của in_proj_weight và qkv Linear cũng khác. Seed không được reset trước từng mô hình. Do vậy các chênh lệch được ghi nhận chỉ phản ánh hai cấu hình đã chạy, chưa chứng minh khác biệt do cách hiện thực MSA và càng chưa phải kiểm thử tương đương số học. Muốn kiểm thử cần sao chép cùng trọng số, tắt dropout khi đối chiếu và kiểm tra đầu ra/gradient [3], [4].

Trong ma trận custom_patch, 41 mẫu số 9 bị nhận thành 4 và 39 mẫu số 3 thành 5; với custom_row, 27 mẫu số 4 thành 9 và 17 mẫu số 7 thành 2 (xem Phụ lục A). Hình lỗi dưới đây cho thấy chữ số có nét biến dạng hoặc ranh giới thị giác mơ hồ. Dữ liệu hiện tại không đủ để quy các lỗi ấy cho một head chú ý cụ thể vì không lưu bản đồ attention của từng ảnh [5].



Hình 4. Một phần mẫu dự đoán sai của custom_patch (10 mẫu đầu trong danh sách lỗi gốc).

CHƯƠNG 4. E3 – LSTM VÀ GRU TRÊN CHUỖI ẢNH

4.1. Biến đổi ảnh thành chuỗi

E3 xem ảnh MNIST như chuỗi có thứ tự thay vì xem từng pixel độc lập. Biểu diễn theo hàng lấy mỗi hàng 28 pixel làm một bước thời gian, nên T = 28 và D = 28. Biểu diễn theo ô lấy các patch 7×7 không chồng lấn, quét từ trái sang phải rồi từ trên xuống dưới, nên T = 16 và D = 49. Cả hai biến đổi chỉ thay đổi cách tổ chức tensor, không tăng số lượng mẫu dữ liệu [4].

Mỗi mô hình có một tầng LSTM hoặc GRU một chiều với hidden size H = 128. Mô hình lấy output[:, −1, :] ở bước cuối rồi qua Dropout(0,1) và Linear(128, 10). Vì num_layers = 1 nên dropout nội bộ của nn.LSTM/nn.GRU được đặt bằng 0; dropout hiệu lực nằm ở head phân loại. Hai mô hình đối chứng mlp_baseline và cnn_baseline gọi lại hàm xây dựng kiến trúc E1 nhưng tạo mới và huấn luyện lại trong E3 [4].

4.2. Trạng thái ẩn, cơ chế cổng và số tham số

LSTM có trạng thái ô nhớ c_t cùng trạng thái ẩn h_t. Cổng quên f_t quyết định phần thông tin cũ được giữ, cổng vào i_t điều khiển thông tin mới, còn cổng ra o_t điều tiết trạng thái quan sát: $c_t = f_t \odot c_{t-1} + i_t \odot g_t$; $h_t = o_t \odot \tanh(c_t)$. GRU không duy trì một vector ô nhớ tách biệt; nó dùng cổng cập nhật và cổng đặt lại để phối hợp trạng thái trước với trạng thái ứng viên. Đây là khác biệt cấu trúc, không phải bằng chứng rằng mô hình nào luôn bắt phụ thuộc dài tốt hơn trên mọi bài toán [4], [11].

Với cách đếm tham số PyTorch gồm hai vector bias cho mỗi lớp hồi quy, LSTM một chiều một tầng có 4H(D + H + 2) tham số ở khối hồi quy, còn GRU có 3H(D + H + 2); head thêm H×10+10 = 1.290. Do đó với D = 28, LSTM có 82.186 và GRU có 61.962 tham số; với D = 49, các con số tương ứng là 92.938 và 70.026. Như vậy GRU ít tham số hơn trong cùng cách biểu diễn, song không bảo đảm thời gian chạy ngắn hơn vì hiệu quả kernel phụ thuộc môi trường [4], [5].

4.3. Kết quả LSTM, GRU và các mô hình đối chứng

Bảng 6. Kết quả sáu cấu hình E3

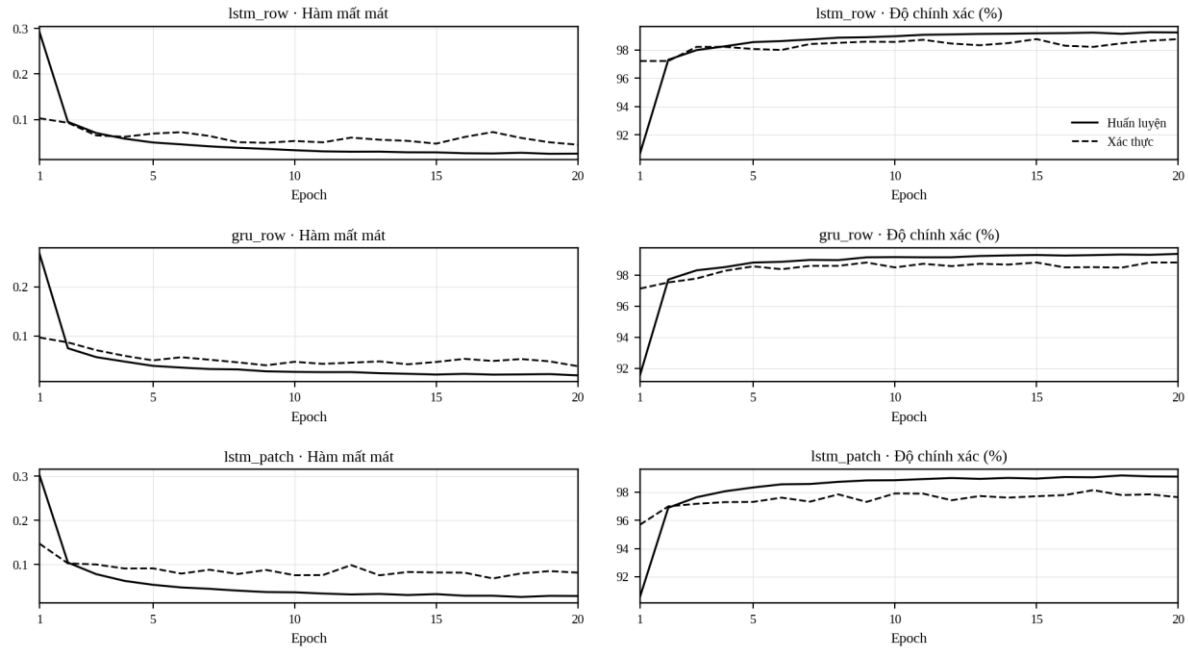
Mô hình	Test accuracy	Macro-F1	Tham số	Huấn luyện	Epoch tốt nhất
LSTM · hàng	98,69%	0,986827	82.186	497,0 s	15

Mô hình	Test accuracy	Macro-F1	Tham số	Huấn luyện	Epoch tốt nhất
GRU · hàng	98,87%	0,988574	61.962	516,6 s	9
LSTM · patch	97,98%	0,979603	92.938	533,4 s	17
GRU · patch	97,84%	0,978211	70.026	535,9 s	20
MLP đối chứng	97,95%	0,979267	235.146	496,6 s	19
CNN đối chứng	99,20%	0,991939	421.642	543,8 s	15

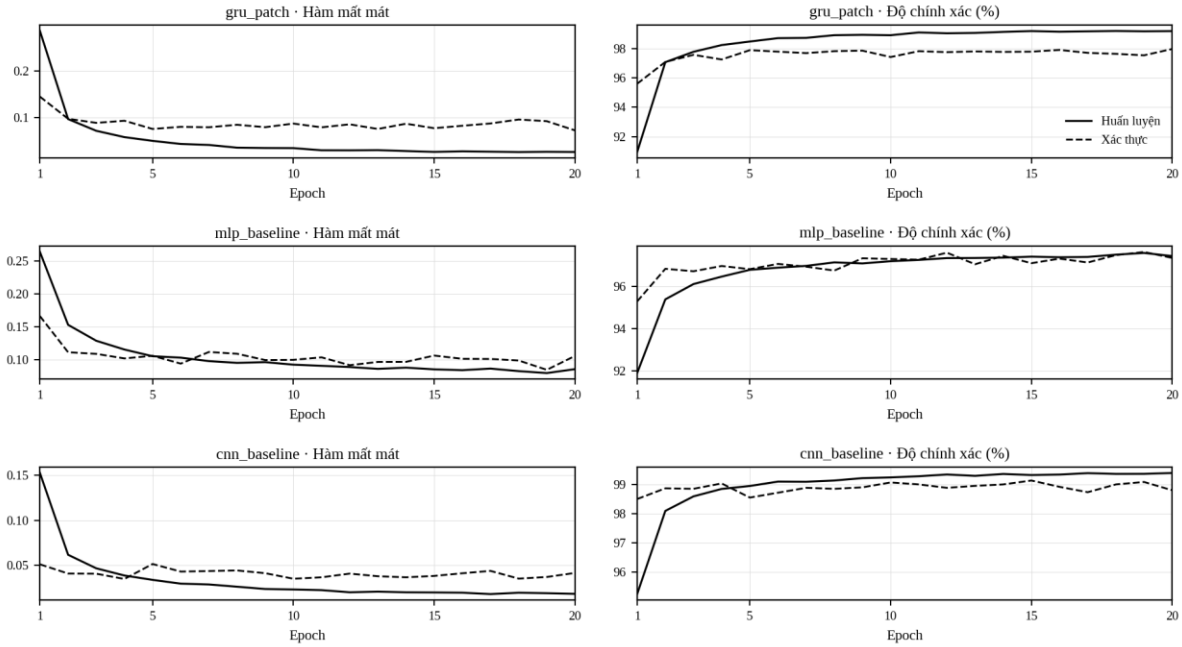
Theo hàng, GRU đạt 98,87%, hơn LSTM 98,69% là 0,18 điểm phần trăm; trong khi GRU chỉ dùng 61.962 so với 82.186 tham số. Tuy vậy, thời gian huấn luyện GRU row ghi nhận 516,6 giây, dài hơn LSTM row 497,0 giây. Khi biểu diễn patch, LSTM đạt 97,98% và GRU đạt 97,84%; so với phiên bản theo hàng, lần lượt giảm 0,71 và 1,03 điểm phần trăm [5].

Trong hai mô hình đối chứng, MLP đạt 97,95%, còn CNN đạt 99,20%. CNN hơn GRU row 0,33 điểm phần trăm, nhưng cần 421.642 tham số, lớn hơn đáng kể 61.962 tham số của GRU row. Chênh lệch 99,20% (CNN E3) và 99,17% (CNN E1) chỉ là hai lần huấn luyện mới khác nhau; không đủ bằng chứng kết luận có cải thiện có ý nghĩa thống kê [5].

4.4. Đường cong và lỗi dự đoán



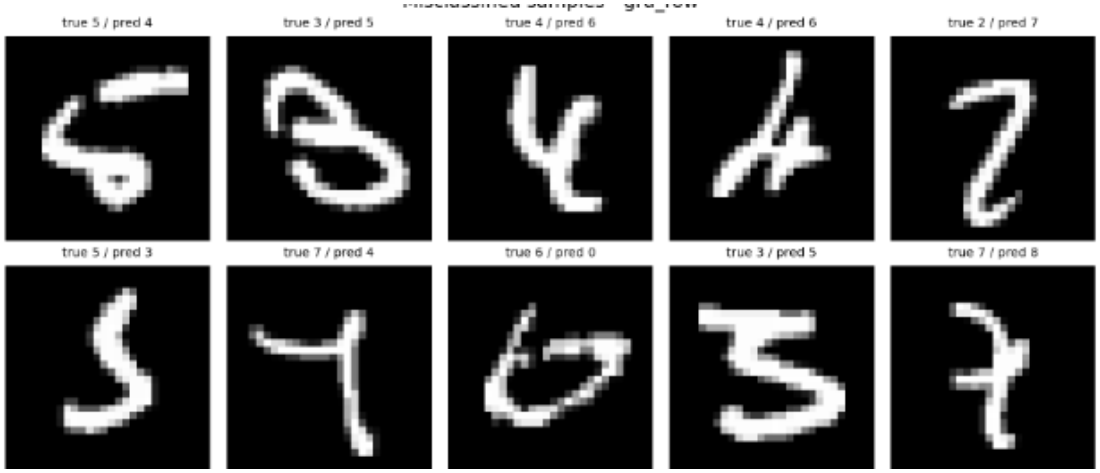
Hình 5. Đường cong E3: LSTM theo hàng, GRU theo hàng và LSTM theo patch, nguồn [6].



Hình 6. Đường cong E3: GRU theo patch cùng MLP/CNN đối chứng, nguồn [6].

Các kiến trúc hồi quy giảm loss nhanh trong vài epoch đầu rồi hội tụ tương đối ổn định. Phần lớn mô hình có đường loss huấn luyện thấp hơn xác thực về cuối, phản ánh khoảng cách khái quát hóa trong lần chạy. Best epoch không nhất thiết là epoch cuối: GRU row đạt mức validation accuracy tốt nhất lần đầu ở epoch 9, LSTM row ở epoch 15; phải đối chiếu giá trị đầy đủ trong lịch sử vì vài epoch có thể hiển thị bằng nhau sau làm tròn [5], [6].

Hình lỗi và ma trận nhầm lẫn E3 (Phụ lục A) vẫn cho thấy sự nhầm lẫn giữa các nét viết gần nhau. Đối với GRU row, nhóm số 5 bị dự đoán thành 3 có 9 mẫu; với CNN đối chứng, nhầm số 9 thành 4 có 9 mẫu. Mặc dù mô hình hồi quy quét tuần tự từ hàng/patch trước sang sau, thí nghiệm không có bài kiểm tra riêng về phụ thuộc dài, nên không thể kết luận định lượng về khả năng ghi nhớ dài hạn từ accuracy MNIST [5].



Hình 7. Mười mẫu lỗi đầu của GRU theo hàng được lấy từ hình gốc, không chọn ngẫu nhiên [4].

CHƯƠNG 5. SO SÁNH TỔNG HỢP VÀ THẢO LUẬN

5.1. Đối chiếu toàn bộ 12 cấu hình

Bảng 7. Tổng hợp chỉ số chất lượng trên tập kiểm tra

Bài	Mã thí nghiệm	Accuracy	Macro-F1	Test loss
E1	softmax	92,09%	0,919632	0,29577
E1	mlp	97,82%	0,978016	0,07753
E1	cnn	99,17%	0,991646	0,02794
E2	custom_patch	95,63%	0,955523	0,14395
E2	pytorch_patch	96,66%	0,966205	0,10740
E2	custom_row	97,40%	0,973801	0,08666
E3	lstm_row	98,69%	0,986827	0,04165
E3	gru_row	98,87%	0,988574	0,03685
E3	lstm_patch	97,98%	0,979603	0,07134
E3	gru_patch	97,84%	0,978211	0,07376
E3	mlp_baseline	97,95%	0,979267	0,07624
E3	cnn_baseline	99,20%	0,991939	0,02906

Bảng tổng hợp cho thấy trong nhóm E1, CNN dẫn đầu; trong E2, custom_row dẫn đầu; trong E3, CNN đối chứng có accuracy cao nhất, tiếp theo là GRU row và LSTM row. Sự khác nhau về mục tiêu từng bài cần được tôn trọng: E1 khảo sát mức độ tận dụng đặc trưng không gian, E2 kiểm tra cách hiện thực và token hóa của attention, còn E3 đánh giá cách diễn giải ảnh thành chuỗi. Vì vậy không nên dùng một bảng accuracy duy nhất để phủ nhận đóng góp về tính chất kiến trúc của các bài [1]–[5].

5.2. Hiệu năng so với chi phí mô hình

Bảng 8. Quy mô tham số và thời gian huấn luyện ghi nhận

Bài	Mã thí nghiệm	Tham số huấn luyện	Thời gian train + val
E1	softmax	7.850	418,4 s
E1	mlp	235.146	471,3 s
E1	cnn	421.642	504,7 s
E2	custom_patch	38.602	660,6 s
E2	pytorch_patch	38.602	650,8 s
E2	custom_row	38.026	637,4 s
E3	lstm_row	82.186	497,0 s
E3	gru_row	61.962	516,6 s
E3	lstm_patch	92.938	533,4 s
E3	gru_patch	70.026	535,9 s
E3	mlp_baseline	235.146	496,6 s
E3	cnn_baseline	421.642	543,8 s

Chỉ nhìn vào số tham số, GRU row dùng khoảng 14,70% số tham số của CNN trong E3 nhưng accuracy thấp hơn 0,33 điểm phần trăm. Tuy nhiên, nó không huấn luyện nhanh hơn mọi cấu hình khác: E3 GRU row (516,6 giây) lâu hơn LSTM row (497,0 giây). E2 gồm các mạng ít hơn 39 nghìn tham số nhưng mất khoảng 637–661 giây, vượt thời gian các mô hình E1 lớn hơn nhiều. Nguyên nhân có thể liên quan đến chi phí vận hành các phép toán chuỗi/attention trên mini-batch nhỏ; hồ sơ chưa có profiling để xác định đóng góp của từng kernel. Thời gian đo không phải thời gian suy luận và không thể coi như kết quả benchmark phần cứng tiêu chuẩn [2]–[5].

5.3. Diễn giải kết quả theo giả thuyết kiến trúc

Softmax ánh xạ trực tiếp pixel sang lớp nên thiếu khả năng học các tổ hợp nét phi tuyến; MLP khắc phục phần nào bằng tầng ẩn, trong khi CNN có phép tích chập chia sẻ trọng số và cơ chế tổng hợp cục bộ. Trên MNIST, điều này phù hợp với quan sát CNN có độ chính xác cao hơn, nhưng vẫn là diễn giải theo inductive bias của kiến trúc, không phải thí nghiệm ablation đã tách riêng từng thành phần [7], [8].

Ở E2, biểu diễn theo hàng duy trì thứ tự hình học theo chiều dọc với 28 token; biểu diễn patch chia ảnh thành 16 vùng 7×7. Chênh lệch accuracy custom_row so với custom_patch có thể gợi ý ảnh hưởng của đơn vị token đối với chữ số nhỏ, nhưng hai cấu hình đồng thời thay đổi lớp projection, độ dài chuỗi và số tham số; không được quy toàn bộ chênh lệch cho riêng độ dài chuỗi. Ở E3, phép quét theo hàng cho kết quả cao hơn patch đối với cả LSTM và GRU. Hai họ mô hình cùng cho tín hiệu phù hợp về vai trò của cách biểu diễn, nhưng không chứng minh hiệu ứng trên tập ảnh khác [3]–[5].

5.4. Hạn chế và cải tiến thực nghiệm

Các số liệu mới chỉ có một seed và một cấu hình tối ưu hóa chung, chưa báo cáo nhiều lần chạy, độ lệch chuẩn, khoảng tin cậy hoặc kiểm định ý nghĩa thống kê. Chưa có bản ghi tên GPU, phiên bản CUDA/PyTorch, bộ đo suy luận (latency/throughput) hay memory profiling; kết quả thời gian chỉ so sánh thô trong môi trường đã chạy. Mã chưa có thử nghiệm tách riêng dropout, khởi tạo, cách lấy đặc trưng cuối, thay đổi số tầng hoặc patch size; do đó chưa đủ nền tảng cho suy luận nhân quả. Hơn nữa, MNIST

là bộ dữ liệu tương đối đơn giản, không đại diện cho các bài toán thị giác phức tạp [2]–[6].

Các bước kiểm chứng tiếp theo có giá trị gồm chạy nhiều seed cố định, giữ cấu hình khởi tạo tương đồng khi so MSA tự viết với PyTorch, so khớp forward/backward với cùng trọng số, đo thời gian suy luận trên cùng môi trường, và thử bộ dữ liệu khó hơn hoặc thí nghiệm riêng về cấu trúc phụ thuộc dài. Đây là khuyến nghị phương pháp, chưa phải thí nghiệm đã thực hiện trong bộ hồ sơ hiện tại.

KẾT LUẬN

Bộ thực nghiệm E1–E3 đã thể hiện ba cách tiếp cận phân loại ảnh: mô hình tuyến tính và học biểu diễn bằng mạng tích chập; mô hình attention trên chuỗi token; và mạng hồi quy trên chuỗi hàng/patch. Các bằng chứng mã nguồn xác nhận vòng lặp huấn luyện tự viết, phân chia dữ liệu nhất quán và ghi nhận 12 lượt chạy với lịch sử đầy đủ. Với cấu hình đã dùng, CNN E1 đạt 99,17%, MSA theo hàng E2 đạt 97,40%, GRU theo hàng E3 đạt 98,87%, và CNN đối chứng chạy lại E3 đạt 99,20% [2]–[6].

Ý nghĩa chính của báo cáo là liên hệ kết quả định lượng với giả thiết của từng kiến trúc, chỉ rõ chi phí và các dạng lỗi, đồng thời giữ ranh giới suy luận. Không có cơ sở từ một bộ MNIST và một seed để khẳng định một họ mô hình luôn tốt hơn, cũng chưa thể xem chênh lệch E2 là bằng chứng hai hiện thực MSA tương đương về mặt số học. Kết quả này nên được hiểu là một thí nghiệm học thuật có thể tiếp tục đối chiếu và tái lập, không phải đánh giá chuẩn hóa toàn diện giữa các họ mạng.

TÀI LIỆU THAM KHẢO

[1] Trường Đại học Bách khoa – ĐHQG TP.HCM. Bài tập E1–E3, học phần CO5085, Học kỳ 261. Tệp 02_DE_BAI_GOC_E1_E3.pdf, 6 trang.

[2] Mã nguồn nghiên cứu cung cấp: common/data.py, common/engine.py, common/plots.py, common/utils.py và E1/models.py, E1/train.py. Tệp 05_MA_NGUON_E1_E3.txt.

[3] Mã nguồn E2/models.py và E2/train.py, trong 05_MA_NGUON_E1_E3.txt.

[4] Mã nguồn E3/models.py và E3/train.py, trong 05_MA_NGUON_E1_E3.txt.

[5] Dữ liệu thực nghiệm: 06_METRICS_GOC_GOP.json và 07_KET_QUA_TONG_HOP.csv, gồm 12 lượt chạy.

[6] Lịch sử thực nghiệm: 08_LICH_SU_HUAN_LUYEN.csv, gồm 240 bản ghi epoch.

[7] Slide học phần: 02-foundation/05-Losses.pdf (tr. vật lý 21–23), 02-foundation/08-MLP.pdf (tr. 7, 11–16), bản trích 09_TRICH_XUAT_SLIDE_LIEN_QUAN.txt.

[8] Slide học phần: 03-cnn/01-Conv2d-Overview.pdf (tr. vật lý 4–5, 39), 03-cnn/06-Pool-Dropout.pdf (tr. 4–5), bản trích 09.

[9] Slide học phần: 05-transformers/03-Model-Families-BERT-GPT-T5-ViT.pdf (tr. vật lý 29–32), bản trích 09.

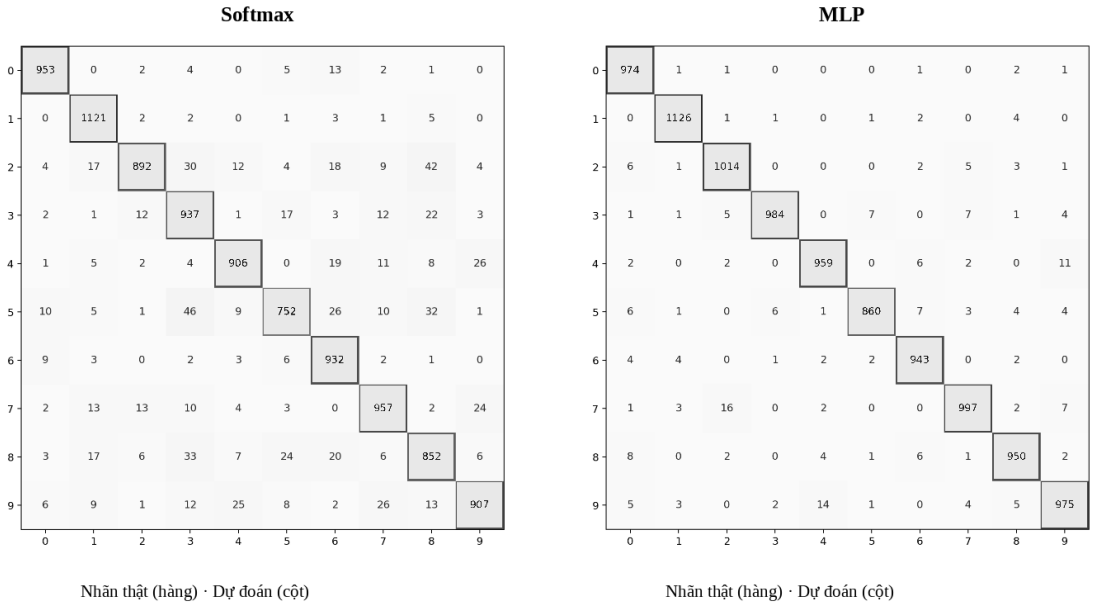
[10] Slide học phần: 05-transformers/02-Attention-Transformer-Core.pdf (tr. vật lý 11–12, 17–23), bản trích 09.

[11] Slide học phần: 04-rnn/RNN-Summary.pdf (tr. vật lý 24–34, 37–39), bản trích 09.

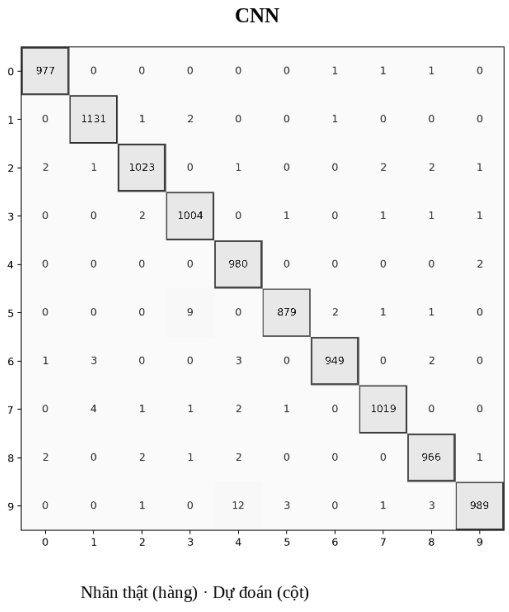
Ghi chú nguồn: Các đường dẫn slide được ghi đúng như tệp trích xuất người học cung cấp. Báo cáo không bổ sung tác giả, năm xuất bản hoặc DOI khi chưa có căn cứ. Toàn bộ hình ma trận và ví dụ lỗi được trích từ tệp 13–18; các đường cong được vẽ lại từ tệp 08.

PHỤ LỤC A. MINH CHỨNG MA TRẬN NHẦM LẪN

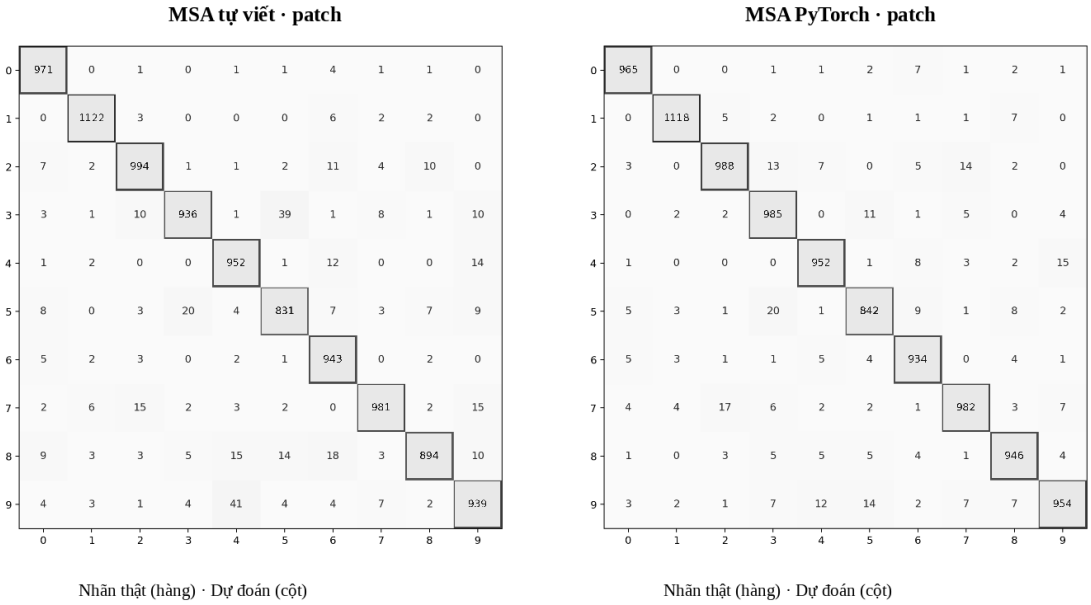
Các hình dưới đây trích từ ma trận nhầm lẫn gốc ở ba bài tập (13_E1_MA_TRAN_NHAM_LAN.png, 14_E2_MA_TRAN_NHAM_LAN.png và 15_E3_MA_TRAN_NHAM_LAN.png). Hàng là nhãn thật, cột là nhãn dự đoán; đường chéo chính là dự đoán đúng. Ảnh giữ nguyên số ô gốc, không tạo lại các giá trị ngoài tệp. Do hạn chế nhãn tiếng Anh có sẵn ở hình nguồn, chú thích tiếng Việt đóng vai trò diễn giải chúng.



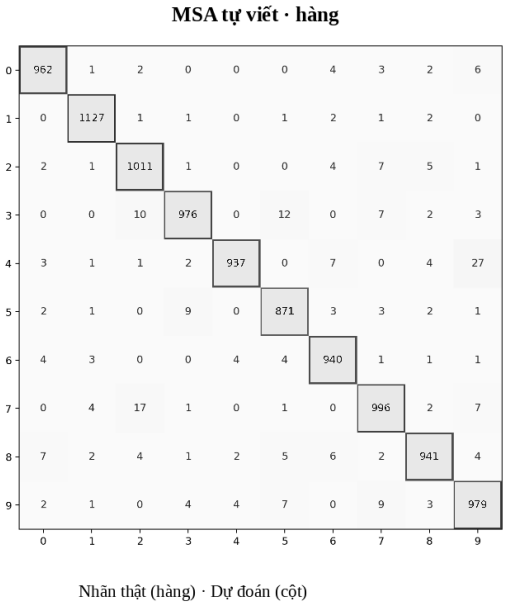
Hình 8. Ma trận nhầm lẫn E1: Softmax (trái) và MLP (phải).



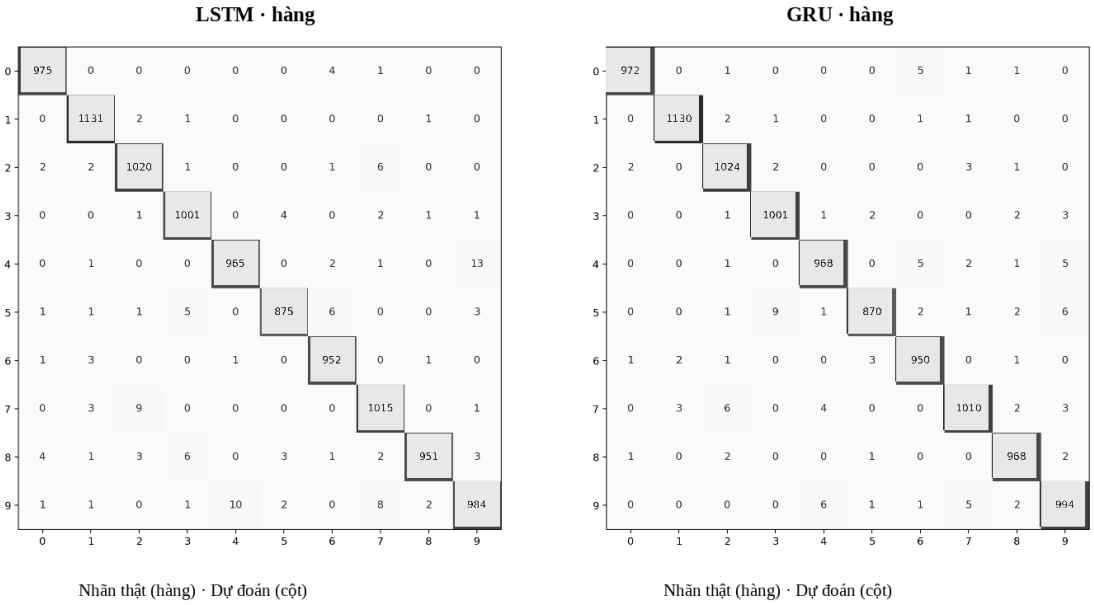
Hình 9. Ma trận nhầm lẫn E1: CNN.



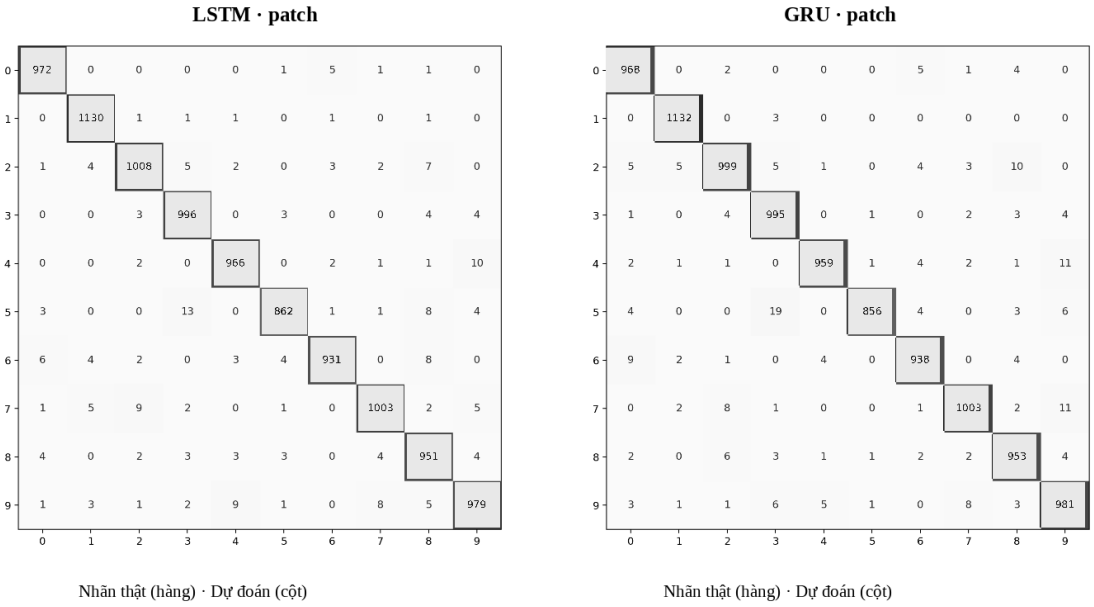
Hình 10. Ma trận nhầm lẫn E2: MSA tự viết theo patch và MSA PyTorch theo patch.



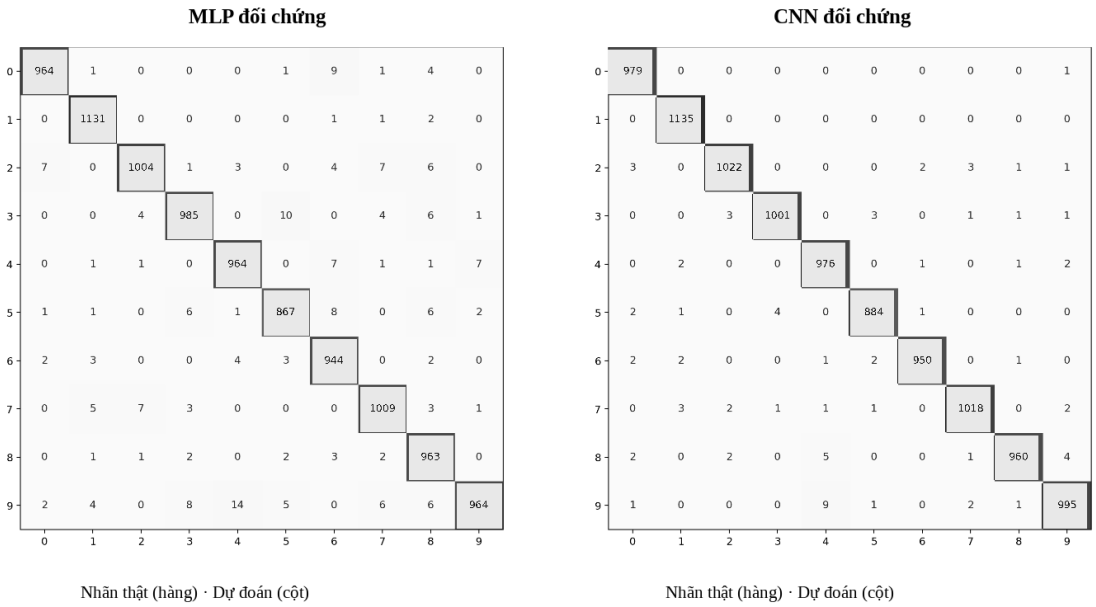
Hình 11. Ma trận nhầm lẫn E2: MSA tự viết theo hàng.



Hình 12. Ma trận nhầm lẫn E3: LSTM theo hàng và GRU theo hàng.



Hình 13. Ma trận nhầm lẫn E3: LSTM theo patch và GRU theo patch.



Hình 14. Ma trận nhầm lẫn E3: MLP và CNN đối chứng được huấn luyện lại.

PHỤ LỤC B. TÁI LẬP VÀ KHAI BÁO HỖ TRỢ AI

B.1. Lệnh gợi ý tái lập theo cấu hình đã lưu

Các dòng lệnh dưới đây được suy ra từ cấu hình trong JSON và tham số CLI của mã nguồn, không được coi là bản ghi nguyên văn lệnh đã dùng ở phiên huấn luyện gốc. Khi đổi phần cứng, phiên bản thư viện hoặc thứ tự chạy, số liệu có thể biến động [2]–[5].

Lệnh 1: python E1/train.py --models all --epochs 20 --batch-size 16 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --device cuda

Lệnh 2: python E2/train.py --experiments all --epochs 20 --batch-size 16 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --embed-dim 64 --num-heads 4 --ffn-dim 128 --depth 1 --patch-size 7 --dropout 0.1 --device cuda

Lệnh 3: python E3/train.py --experiments all --epochs 20 --batch-size 16 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --hidden-dim 128 --num-layers 1 --patch-size 7 --dropout 0.1 --device cuda

B.2. Các điểm cần xác thực trong bước hoàn thiện

Người học cần đối chiếu báo cáo với repo hiện hành trước khi nộp, kiểm tra tên môi trường/GPU nếu muốn bổ sung và xác minh hình khoa học sau khi cập nhật. Bộ tài liệu hiện có không ghi tệp checkpoint đã lưu ra đĩa, toàn bộ nhãn dự đoán dưới dạng bảng số hoặc nhiều seed; báo cáo không khẳng định các minh chứng này đã tồn tại.

B.3. Khai báo sử dụng công cụ AI

Trong quá trình chuẩn bị bản nháp, công cụ AI được sử dụng để hỗ trợ tổ chức tài liệu, đối chiếu yêu cầu với mã nguồn và số liệu, soạn thảo phần diễn giải ban đầu và bố trí bảng/hình trong Word. Đây là phạm vi hỗ trợ soạn thảo thực tế, không chỉ là kiểm tra chính tả. Theo quy định liêm chính học thuật của học phần, người học có trách nhiệm tự kiểm tra, chỉnh lý đáng kể bản nháp, xác nhận toàn bộ kết quả, giải thích được mã nguồn và bảo vệ các nhận định trước khi sử dụng nội dung để nộp bài [1].